

Technical Documentation: CNN for 5G NR Channel Estimation

Thien Huynh-The

September 21, 2025

Abstract

The general objective of this codebase is to build, train, and evaluate a Convolutional Neural Network (CNN) to perform channel estimation in a 5G NR wireless communication system.

1 Core Concept

The core idea is as follows:

- **Input:** Provide the CNN with a preliminary channel estimate, generated using a simple linear interpolation method from the Demodulation Reference Symbols (DM-RS).
- **Label:** Provide the network with the correct answer – the perfect channel estimation obtained from the simulated environment.
- **Training:** The CNN learns to correct errors and denoise the preliminary estimate from the input to produce a result that is as close as possible to the perfect channel response (the label).
- **Evaluation:** Compare the performance (MSE - Mean Squared Error) of the CNN against other traditional methods: linear interpolation and a practical estimation algorithm.

2 Part 1: Training Data Generation (`hGenerateTrainingData.m`)

This process generates 1024 pairs of input data samples and their corresponding labels. Each sample is created through an independent simulation cycle with randomized channel conditions to ensure data diversity.

2.1 5G NR Simulation Configuration (`hDeepLearningChanEstSimParameters.m`)

The fundamental technical parameters for the simulation are defined as follows:

- **Carrier:**
 - Bandwidth: 51 Resource Blocks (RBs).
 - Subcarrier Spacing: 30 kHz.
 - Cyclic Prefix: Normal.
 - Antenna Configuration: SISO (Single-Input Single-Output) with 1 transmit and 1 receive antenna.
- **PDSCH and DM-RS:**
 - Modulation: Fixed at 16QAM for the entire training data generation process.

- DM-RS Position: The first DM-RS symbol is at position 2 (`DMRSTypeAPosition = 2`).
- DM-RS Length: 1 symbol (`DMRSLength = 1`).
- DM-RS Configuration Type: Type 2 (`DMRSConfigurationType = 2`).

2.2 Detailed Process of Generating a Single Data Sample

Let's imagine we are at an arbitrary iteration `i` within the loop for `i = 1:numExamples`. The following steps occur to create the pair `trainData(:, :, :, i)` and `trainLabels(:, :, :, i)`.

1. **Step 1: “Rolling the Dice” - Creating a Unique Channel Environment** Each time the loop runs, a completely new and unique propagation environment is created.

- `channel.release`: This command releases the properties of the channel object, allowing us to change non-tunable parameters during runtime (like `DelayProfile`).
- `channel.Seed = randi([1001 2000])`: A new random seed is assigned to the channel. This ensures that the random variations of the channel (fading) will be different in each iteration.
- `channel.DelayProfile = string(delayProfiles(randi([1 numel(delayProfiles)]))))`: Randomly selects one of the 5 channel delay profiles (“TDL-A” to “TDL-E”). Each profile simulates a different propagation environment (e.g., urban, rural) with different multipath characteristics.
- `channel.DelaySpread = randi([1 300])*1e-9`: Assigns a random delay spread value, which determines the time difference between multipath components.
- `channel.MaximumDopplerShift = randi([5 400])`: Assigns a random Doppler shift value, simulating the relative speed between the transmitter and receiver.

Result of Step 1: We have a `channel` object configured with a completely random and unique set of propagation characteristics for this iteration `i`.

2. **Step 2: Transmitting the Signal and Observing Channel Effects** The original (unchanging) reference signal is transmitted through the newly created channel environment.

- `txWaveform = [txWaveform_original; zeros(maxChDelay, ...)]`: The original OFDM signal `txWaveform_original` (containing only DM-RS) is padded with zeros. This padding ensures that the entire signal, including delayed components due to multipath, passes through the channel filter.
- `[rxWaveform, pathGains, sampleTimes] = channel(txWaveform)`: The `txWaveform` is “transmitted” through the `channel` object. `rxWaveform` is the received waveform, distorted by the channel’s fading and multipath effects. `pathGains` contains perfect information about the gain and phase of each path in the channel, which will be used to create the “label”.

Result of Step 2: We have `rxWaveform`, the realistic version of the signal after being impaired by the channel environment.

3. **Step 3: Simulating Additive Noise** A realistic environment always includes noise.

- `SNRdB = randi([0 10])`: A random SNR level from 0 to 10 dB is chosen.
- The SNR is converted to a linear scale, and the noise power (`N0`) is calculated.
- A complex Gaussian noise vector is generated and added to the received waveform.

Result of Step 3: `rxWaveform` is now the most realistic version of the signal that the receiver can obtain, impaired by both the channel and noise.

4. **Step 4: “Reverse Translating” - Reconstructing the Resource Grid** The receiver processes `rxWaveform` to recover the frequency-domain data.

- `[offset, ] = nrPerfectTimingEstimate(pathGains,pathFilters)`: Uses perfect channel information to find the exact start of the signal in `rxWaveform` (an assumption of “perfect synchronization”).
- The received waveform is trimmed based on the estimated offset.
- `rxGrid = nrOFDMDemodulate(carrier,rxWaveform)`: OFDM demodulation is performed to transform the signal from the time domain back to the frequency domain, reconstructing the resource grid `rxGrid`.

Result of Step 4: We have `rxGrid`, a 2D “image” of the channel in the frequency-time domain, but this “image” is blurred and noisy.

5. **Step 5 & 6: Creating the Golden Pair - “Blurred Image” and “Original Image”** From `rxGrid` and the perfect channel information, we create the training data pair.

- **Step 5: Create the Label (Original Image)** `estChannelGridPerfect = nrPerfectChannelEst`
This function uses the perfect `pathGains` from Step 2 to reconstruct the channel response across the entire resource grid with absolute precision. This is the ideal, noise-free version. This complex grid is split into its real and imaginary parts and stored as `trainLabels(:, :, :, i)`.
- **Step 6: Create the Input (“Blurred Image”)** The received values at the DM-RS locations are extracted from `rxGrid`. A preliminary channel estimate is performed only at these DM-RS points. Then, a scattered interpolant object is used to estimate the channel values across the entire grid, essentially filling in the blanks between the DM-RS points using linear interpolation. The result, `hest`, is an inaccurate and noisy full channel estimate. This complex grid is split into its real and imaginary parts and stored as `trainData(:, :, :, i)`.

3 Part 2: CNN Architecture and Training (`guide_runningProgram.m`)

3.1 CNN Architecture

The network is constructed from the following layers in sequence:

- `imageInputLayer`: Input layer accepting an image of size 612x14x1.
- `convolution2dLayer`: 2D convolutional layer with a 9x9 filter size and 64 filters.
- `reluLayer`: ReLU activation layer.
- `convolution2dLayer`: 2D convolutional layer with a 5x5 filter size and 64 filters.
- `reluLayer`: Activation layer.
- `convolution2dLayer`: 2D convolutional layer with a 5x5 filter size and 64 filters.
- `reluLayer`: Activation layer.
- `convolution2dLayer`: 2D convolutional layer with a 5x5 filter size and 32 filters.
- `reluLayer`: Activation layer.

- **convolution2dLayer:** Final 2D convolutional layer with a 5x5 filter size and 1 filter, to return the output to the original dimensions.
- **regressionLayer:** Regression layer, which defines the loss function as Mean Squared Error (MSE), suitable for the task of predicting channel values.

3.2 Training Parameters and Process

Training is configured with the following parameters:

- **Optimizer:** Adam (adam).
- **Initial Learn Rate:** 1×10^{-4} .
- **Max Epochs:** 20.
- **Mini-Batch Size:** 32 samples per weight update.
- **Shuffle:** Data is shuffled at every epoch (every-epoch).
- **Validation Data:** 25% of the initial data is used to check the model's performance during training.
- **Validation Patience:** Training will stop early if performance on the validation set does not improve after 5 consecutive checks.

Finally, the `trainNetwork` function is called to begin the training process.

4 Part 3: Quick Evaluation with a Single Sample

The objective of this section is to take the trained CNN and directly compare its performance against traditional channel estimation methods on a single, non-random data sample.

1. **Step 1: Setup a Fixed Simulation Environment** Unlike training data generation, here we create a single, deterministic channel environment for a fair comparison.
 - A TDL channel is initialized with fixed properties: Seed = 0, Delay Profile = 'TDL-A', Delay Spread = 300 ns, Maximum Doppler Shift = 50 Hz.
 - The SNR is fixed at SNRdB = 10.
2. **Step 2: Simulate Signal Transmission and Reception** A reference signal is passed through the fixed channel environment. Noise is added, and the receiver performs perfect synchronization and OFDM demodulation to reconstruct the received grid, `rxGrid`.
3. **Step 3: Perform Four Channel Estimation Methods Concurrently** From `rxGrid`, four different channel estimation methods are performed:
 - **Perfect Channel Estimation (Ground Truth):** Uses foreknowledge from the simulation to generate the absolutely correct channel response. This serves as the answer key.
 - **Practical Estimator:** Uses MATLAB's standard `nrChannelEstimate` function, which is a realistic baseline based on algorithms like Least Squares.
 - **Linear Interpolation Estimator:** Performs simple linear interpolation from the DM-RS points. This is the same method used to generate the CNN's input.

- **CNN Estimator:** The interpolated channel grid is fed into the trained CNN (`channelEstimationCNN`), and the `predict` function is called to get the network's estimate.
4. **Step 4: Calculate Mean Squared Error (MSE)** The MSE is calculated for the three practical methods by comparing their outputs to the perfect channel estimate.
 5. **Step 5: Visualize the Results** A plotting function is called to generate a figure with four subplots. Each subplot displays the magnitude of a channel grid, allowing for a visual comparison of the shape of the estimated channel versus the actual channel. The calculated MSE value is displayed in the title of each respective plot.

5 Part 4: Systematic Evaluation

5.1 Phase 2: Systematic Test Data Generation (`testDataGeneration.m`)

Unlike training data generation, which prioritizes randomness, the goal here is to create a structured and controlled dataset. This allows for a scientific evaluation of the model's performance under specific conditions (e.g., how well does the model perform at low SNR with high-order modulation?).

- **Modified Functions:** Two helper functions are modified (`hDeepLearningChanEstSimParameters.m` and `hGenerateTrainingData.m`) to accept `modulation` and `snr` as input arguments, allowing for controlled data generation.
- **Data Generation Process:** The script defines test scenarios with a range of SNRs (`snrVec = [-10 0 10 20 30]`) and modulations (`modVec = {'QPSK', '16QAM', '64QAM', '256QAM'}`). It uses nested loops to iterate through every possible combination. For each combination, it generates 512 samples and saves the outputs of all estimation methods (perfect, practical, interpolated) into large arrays.
- **Data Storage:** After all loops are complete, the entire dataset is saved to a single large file, `testData.mat`, using the `-v7.3` flag to handle its large size.

5.2 Phase 3: System Performance Evaluation (`testing_program.m`)

This phase uses the dataset from Phase 2 to perform a detailed performance test of the trained CNN.

- **Preparation:** The script loads the trained CNN model from `channelEstimationCNN.mat` and the complete test dataset from `channelEstimationTestData.mat`.
- **Iterative Evaluation Process:** The script loops through every sample in the test dataset. In each iteration, it:
 1. Extracts the data for the current sample (`i`) from the pre-loaded arrays (perfect, practical, interpolated grids, and the CNN input).
 2. Uses the loaded CNN to predict the channel estimate based on the current sample's input.
 3. Calculates the MSE for all three methods (CNN, interpolation, practical) by comparing them to the perfect channel response for that sample.

Outcome of Phase 3: After the loop finishes, the collected MSE data can be used to plot detailed performance graphs (e.g., MSE vs. SNR for each estimation method), leading to a final conclusion about the effectiveness of the CNN model.

6 Evaluation of CNN-Based Channel Estimation

The trained CNN model is evaluated using a structured dataset. The testing pipeline follows these steps:

6.1 Loading the Model and Data

- The trained deep model is loaded from `channelEstimationCNN.mat`.
- The structured test dataset is loaded from `channelEstimationTestData.mat`, which contains:
 1. `perfectGridData` – ground truth channel estimates.
 2. `practicalGridData` – estimates using MATLAB’s practical estimator.
 3. `interpolateGridData` – estimates using linear interpolation.
 4. `nnInputGridData` – CNN input tensors with real and imaginary parts separated.

6.2 Iterative Evaluation

For each test sample i , the following operations are performed:

1. Extract the perfect estimate, practical estimate, interpolation estimate, and CNN input.
2. Run the CNN forward pass:

$$\hat{H}_{\text{CNN}} = \text{predict}(\text{channelEstimationCNN}, \text{nnInput})$$

3. Recombine real and imaginary parts into complex form:

$$\hat{H}_{\text{CNN}} = \Re(\hat{H}_{\text{CNN}}) + j \cdot \Im(\hat{H}_{\text{CNN}})$$

4. Compute the Mean Squared Error (MSE) for each method:

$$\begin{aligned} \text{MSE}_{\text{CNN}} &= \frac{1}{N} \sum_{k=1}^N \left| H_{\text{perfect}}(k) - \hat{H}_{\text{CNN}}(k) \right|^2 \\ \text{MSE}_{\text{interp}} &= \frac{1}{N} \sum_{k=1}^N \left| H_{\text{perfect}}(k) - \hat{H}_{\text{interp}}(k) \right|^2 \\ \text{MSE}_{\text{practical}} &= \frac{1}{N} \sum_{k=1}^N \left| H_{\text{perfect}}(k) - \hat{H}_{\text{practical}}(k) \right|^2 \end{aligned}$$

6.3 Performance Comparison

- The CNN estimator is expected to outperform linear interpolation by reducing noise impact and refining channel estimates.
- Performance relative to the practical estimator depends on channel conditions and SNR.
- To evaluate systematically, MSE values across all test samples should be stored and averaged:

$$\overline{\text{MSE}}_{\text{method}} = \frac{1}{M} \sum_{i=1}^M \text{MSE}_{\text{method}}^{(i)}$$

7 Dataset Description

The dataset `channelEstimationData.mat` is generated for training and validating a deep learning-based channel estimation model. Each sample represents a full resource grid of the wireless channel in the frequency-time domain.

7.1 Dataset Dimensions

The dataset has the shape:

$$612 \times 14 \times 2 \times 40960$$

- 612: number of subcarriers. This corresponds to 51 resource blocks (RBs), where each RB contains 12 subcarriers ($612 = 51 \times 12$).
- 14: number of OFDM symbols in one slot (for normal CP and 15 kHz subcarrier spacing).
- 2: real and imaginary parts of the complex channel coefficients are stored separately.
- 40960: total number of resource grids (samples).

Thus, one resource grid has size $612 \times 14 \times 2$, and the dataset contains 40,960 such grids.

7.2 Data Generation Settings

The samples are generated under different modulation schemes and SNR conditions. Each combination of modulation and SNR produces 2048 resource grids. The detailed allocation is given in Table 1.

Table 1: Dataset composition for `channelEstimationData.mat`

Modulation	SNR (dB)	No. of samples	Index range
QPSK	-10, 0, 10, 20, 30	5×2048	1 – 10240
16QAM	-10, 0, 10, 20, 30	5×2048	10241 – 20480
64QAM	-10, 0, 10, 20, 30	5×2048	20481 – 30720
256QAM	-10, 0, 10, 20, 30	5×2048	30721 – 40960

7.3 Usage for Training and Validation

The dataset is designed for supervised training of a CNN-based channel estimator. A typical split is:

- Training: 80% (32,768 samples).
- Validation: 20% (8,192 samples).

7.4 Evaluation Metric

The model performance is evaluated using the Root Mean Square Error (RMSE) between the predicted channel response $\hat{H}$ and the perfect channel estimate H :

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N |H(i) - \hat{H}(i)|^2}$$

where $N = 612 \times 14$ is the total number of resource elements in one grid.